

Assembler

Синтаксис Intel. Арифметические и логические команды.

Подготовил
Старший преподаватель
МИЭМ НИУ ВШЭ
Морозов Владимир Игоревич

Синтаксис Intel

Основные отличия

- Перед числами и регистрами не нужны специальные символы (\$ и %)
- Перед названиями секций (**.data** и **.text**) нужно слово **section**
- Операция разадресации выглядит как **[]** вместо **()**
- В командах работы с данными не нужно ставить букву, обозначающую размер данных
- В командах работы с данными сначала идёт приёмник, затем источник (например, в **mov**)
- Точка входа называется **_main** (зависит от конкретного ассемблера)

Пример

Синтаксис AT&T

```
.text
```

```
main:
```

```
    movl $0x0, %eax
```

```
    movl (%ebx), %eax
```

Синтаксис Intel

```
section .text
```

```
_main:
```

```
    mov eax, 0
```

```
    mov eax, [ebx]
```

Основные отличия

- При объявлении элементов данных после названия не ставится двоеточие
- Типы данных называются по-другому
- Перед названием типа данных не ставится точка

Названия типов данных

Тип	Директива	Количество байт
Байт	DB	1
Слово	DW	2
Двойное слово	DD	4
8 байт	DQ	8
10 байт	DT	10

Пример

Синтаксис AT&T

```
.data
```

```
    x: .word 0x0
```

```
    y: .byte 0x1
```

Синтаксис Intel

```
section .data
```

```
    x DD 0
```

```
    y DB 1
```

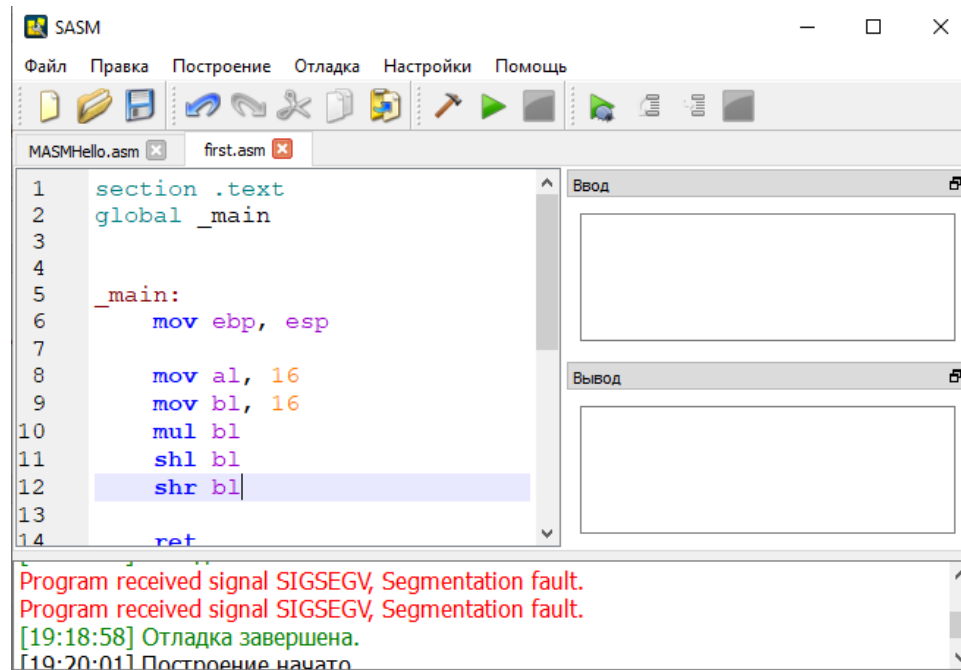
Средства запуска и отладки программ

Средства запуска и отладки

- Есть множество средств для запуска и отладки программ на ассемблере, написанных с использованием синтаксиса Intel
- Наиболее популярные из них:
 - Macro Assembler (MASM) от компании Microsoft
 - Turbo Assembler (TASM) от компании Borland
 - Netwide Assembler (NASM)
 - Flat Assembler (FASM)
- Каждый из представленных ассемблеров обладает своими преимуществами, недостатками, а также особенностями синтаксиса
- В рамках нашего курса будет использоваться NASM

Средства запуска и отладки

- Чтобы разрабатывать и запускать программы на ассемблере наиболее простым способом, можно воспользоваться IDE Simple Assembler (SASM), которая свободно доступна в Интернете



Основы работы с SASM

- В SASM уже выбран NASM в качестве ассемблера по умолчанию
- Чтобы запустить программу, достаточно написать её код в окне кода и нажать кнопку запуска
- Сообщения об ошибках выводятся в окне снизу
- Для отладки (и демонстрации работы программы) следует нажать кнопку запуска с жучком
- Во время отладки на вкладке «Отладка» доступны полезные опции «Показать регистры» и «Показать память». С их помощью можно отследить результат работы программы

Арифметические команды

Сложение

- Сложение может быть выполнено с помощью команды **ADD**
- В качестве приёмника может быть регистр или элемент данных, в качестве источника – регистр, элемент данных или число
- Оба аргумента должны быть одинакового байтового размера

ADD *Приемник, Источник*

$$\langle \text{Приемник} \rangle = \langle \text{Приемник} \rangle + \langle \text{Источник} \rangle$$

Вычитание

- Вычитание может быть выполнено с помощью команды **SUB**
- В качестве приёмника может быть регистр или элемент данных, в качестве источника – регистр, элемент данных или число
- Оба аргумента должны быть одинакового байтового размера

SUB *Приемник, Источник*

<Приемник> = <Приемник> - <Источник>

Инкремент и декремент

- Инкремент и декремент выполняются с помощью команд **INC** и **DEC** соответственно
- В качестве аргумента может быть регистр или элемент данных

INC *Операнд*

DEC *Операнд*

INC: $\langle \text{Операнд} \rangle = \langle \text{Операнд} \rangle + 1$

DEC: $\langle \text{Операнд} \rangle = \langle \text{Операнд} \rangle - 1$

Изменение знака

- Изменение знака – умножение числа на **-1**
- Выполняется с помощью команды **NEG**
- В качестве аргумента может быть регистр или элемент данных

NEG *Операнд*

$$\langle \text{Операнд} \rangle = - \langle \text{Операнд} \rangle$$

Умножение без знака

- Умножает беззнаковые числа
- Выполняется с помощью команды **MUL**
- Первый множитель хранится в заранее определённом регистре, в качестве аргумента передаётся только второй множитель
- **Результат попадает в два регистра** – в один старшая половина, в другой младшая

Умножение без знака

Размер операнда	Множитель	Результат
1 байт	AL	AX
2 байта	AX	DX:AX
4 байта	EAX	EDX:EAX

Умножение со знаком

- Умножает числа со знаком
- Выполняется с помощью команды **IMUL**
- Доступен в трёх вариациях:
 - С указанием только второго множителя (аналогично **MUL**)
 - С указанием обоих множителей (результат записывается в первый)
 - С указанием места под результат, первого и второго множителя (результат и первый множитель должны быть одного размера, второй множитель – только число)

Деление без знака

- Делит беззнаковые числа
- Выполняется с помощью команды **DIV**
- Делимое хранится в заранее определённом регистре (или двух регистрах), в качестве аргумента передаётся только делитель
- **Результат попадает в два регистра** – в один целая часть от деления, в другой остаток

Деление без знака

Размер операнда (делителя)	Делимое	Частное	Остаток
1 байт	AX	AL	AH
2 байта	DX:AX	AX	DX
4 байта	EDX:EAX	EAX	EDX

Деление со знаком

- Делит числа со знаком
- Выполняется с помощью команды **IDIV**
- Остальные особенности аналогичны команде **DIV**

Побитовые команды

Логическое И

- Выполняет операцию «Логическое И» попарно над каждым из битов аргументов
- Выполняется с помощью команды **AND**
- В качестве первого аргумента принимает элемент данных или регистр, в качестве второго – элемент данных, регистр или число
- Результат помещается в первый аргумент

Логическое ИЛИ

- Выполняет операцию «Логическое ИЛИ» попарно над каждым из битов аргументов
- Выполняется с помощью команды **OR**
- В качестве первого аргумента принимает элемент данных или регистр, в качестве второго – элемент данных, регистр или число
- Результат помещается в первый аргумент

Исключающее ИЛИ

- Выполняет операцию «Исключающее ИЛИ» попарно над каждым из битов аргументов
- Выполняется с помощью команды **XOR**
- В качестве первого аргумента принимает элемент данных или регистр, в качестве второго – элемент данных, регистр или число
- Результат помещается в первый аргумент

Логическое отрицание

- Выполняет операцию «Логическое отрицание» над каждым из битов аргумента
- Выполняется с помощью команды **NOT**
- В качестве первого аргумента принимает элемент данных или регистр
- Результат помещается в аргумент

Побитовые сдвиги

- Выполняют перемещение битов двоичного представления данных на нужное количество позиций вправо или влево
- Вытесненные биты уничтожаются
- Новые позиции заполняются нулями

- Пример:

исходные данные:

10001010

побитовый сдвиг на 2 бита вправо:

00100010

Побитовые сдвиги

- Выполняются с помощью:
 - **SHR** – побитовый сдвиг вправо
 - **SHL** – побитовый сдвиг влево
- В качестве первого аргумента принимают элемент данных или регистр, которые необходимо сдвинуть
- В качестве второго аргумента принимают количество позиций сдвига

Дополнительные источники

- <https://www.sites.google.com/site/sistprogr/lekcii1/lek7> – арифметические операторы
- <https://www.sites.google.com/site/sistprogr/lekcii1/lek8> – побитовые операторы
- <https://ravesli.com/uroki-assemblera/> – синтаксис Intel в целом
- <https://ozlib.com/865524/informatika/sdvigi> – подробнее о различных видах побитовых сдвигов